

Entrega 3 – Módulo de Reseñas Dann-Alpes

Nicolás Sánchez - 202420978
Emanuela Martínez - 202420655
Nicolás Castaño –

URL de la API: <https://entrega-proyecto-3.onrender.com/>

Introducción

Este documento presenta las decisiones de diseño que tomamos para el desarrollo de la entrega 3, donde se desarrolla el módulo de reseñas para el sistema hotelero Dann-Alpes. El módulo se apoya en MongoDB como gestor documental, funcionando en paralelo con la base de datos relacional Oracle, que ya se encarga de gestionar las reservas, los clientes y los hoteles.

La elección de MongoDB para las reseñas surge de las características propias de este tipo de dato: el volumen es variable, la estructura puede ser heterogénea entre documentos, ya que no todas las reseñas incluyen respuesta del hotel ni todos los huéspedes califican cada aspecto del servicio, y el patrón de acceso más frecuente consiste en recuperar todas las reseñas de un hotel de una sola vez. Por estas razones, el modelo documental se ajusta mejor a esta entidad que un esquema relacional tradicional.

El documento se organiza en cuatro partes: el análisis de carga de trabajo, el modelo conceptual con las decisiones de diseño, las sentencias de consulta para los requerimientos analíticos, y los escenarios de prueba.

1. Análisis de Carga de Trabajo (Workload)

1.a Volumen estimado de datos

Entidad	Fuente	Registros estimados	Criterio de estimación
HOTEL (Oracle)	Oracle	~20 hoteles	Cadena con presencia en las principales ciudades del país
CLIENTE (Oracle)	Oracle	>50.000 registros	Dato explícito del enunciado
RESERVA (Oracle)	Oracle	~250.000 (acumulado 2 años)	Cada reserva completada es una reseña potencial
reseñas (MongoDB)	MongoDB	~250.000 documentos	300–500 reseñas/día × 730 días, según enunciado
votos_utilidad (MongoDB)	MongoDB	~365.000 documentos	~500 votos/día × 730 días

1.b Operaciones de lectura y escritura (Anexo A)

Entidad(es)	Operación	Información necesaria	Tipo
reseñas	Consultar reseñas de un hotel (RF4)	calificacion, texto, fecha_creacion, votos_utilidad, respuesta_hotel embebida	Lectura
reseñas	Historial de reseñas propias (RF6)	estado, calificacion, respuesta_hotel, votos_utilidad, id_hotel	Lectura
reseñas	Crear reseña (RF1)	Documento completo de reseña	Escritura
reseñas	Editar reseña (RF2)	texto, calificacion, fecha_modificacion	Escritura
reseñas	Eliminar reseña (RF3 / RF8)	estado, fecha_eliminacion, eliminado_por	Escritura
reseñas	Responder reseña (RF7)	Subdocumento respuesta_hotel completo	Escritura
reseñas	Destacar reseña (RF9)	Campo destacada (booleano)	Escritura
votos_utilidad	Votar por una reseña (RF5)	id_resena, id_usuario, fecha_voto	Escritura
reseñas	Top 10 hoteles (RFC1)	Promedio de calificacion agrupado por hotel en periodo	Lectura / Agregación
reseñas	Evolución de reputación (RFC2)	Promedio mensual de calificacion para un hotel en un año	Lectura / Agregación
reseñas	Comparativo por ciudad (RFC3)	Métricas por hotel filtradas por ciudad: promedio, % respuesta, % destacadas	Lectura / Agregación

1.c Cuantificación de operaciones (Anexo B)

Entidad	Operación	Información necesaria	Tipo	Tasa estimada
reseñas	Consultar reseñas de un hotel	calificacion, texto, votos, fecha, respuesta	Lectura	~10.000/día (~7/min)
reseñas	Crear reseña	Documento completo	Escritura	300–500/día
reseñas	Editar reseña	texto, calificacion	Escritura	~100/día
reseñas	Eliminar reseña (cliente/admin)	estado, fecha_eliminacion	Escritura	~50/día
reseñas	Responder / editar respuesta	Subdocumento respuesta_hotel	Escritura	200–300/día
reseñas	Destacar reseña	Campo destacada	Escritura	~20/día
votos_utilidad	Votar por utilidad	id_resena, id_usuario	Escritura	~500/día (~0,35/min)
reseñas	Consultas analíticas RFC1–RFC3	Agregaciones complejas multi-etapa	Lectura	~30/día (gerencia)

La operación más crítica —con diferencia— es la consulta de reseñas por hotel, que representa el 95 % de la carga de lectura. Esto refuerza la decisión de indexar por id_hotel + fecha_creacion y de mantener la respuesta del hotel embebida dentro del mismo documento.

2. Modelo Conceptual y Diseño de Colecciones

2.a Entidades del modelo

Entidad	Descripción	Origen
reseñas	Documento central del módulo. Almacena la calificación (1–5), el texto libre del huésped, el estado (publicada/eliminada), el contador de votos de utilidad, metadatos de fechas, la respuesta oficial del hotel como subdocumento embebido, y el flag de reseña destacada. Es la colección más consultada del sistema.	MongoDB
votos_utilidad	Registra cada acción de marcar una reseña como útil. Se mantiene separada de la reseña para permitir escalar sin límite, aplicar un índice único compuesto que impida votos duplicados, y evitar que el documento de reseña crezca de forma indefinida.	MongoDB
HOTEL	Hotel de la cadena. Sus datos maestros (nombre, dirección, ciudad) viven en Oracle. MongoDB almacena únicamente el id_hotel como referencia.	Oracle
CLIENTE	Huésped registrado en la plataforma. Sus datos personales residen en Oracle. MongoDB referencia su id_cliente.	Oracle
RESERVA	Reserva hotelera. Solo las que tienen estado 'completada' en Oracle pueden dar origen a una reseña en MongoDB (RF1).	Oracle
CIUDAD	Ciudad en la que opera cada hotel. Se usa en la consulta RFC3 para filtrar y comparar hoteles dentro de un mismo mercado. Los ids de ciudad provienen de Oracle vía APEX.	Oracle

2.b Relaciones y cardinalidad

Relación	Cardinalidad	Descripción
CLIENTE → reseñas	1 : N	Un cliente puede escribir múltiples reseñas, una por cada reserva completada distinta.
HOTEL → reseñas	1 : N	Un hotel acumula todas las reseñas que los huéspedes escriben sobre él.
RESERVA → reseñas	1 : 1	Una reserva completada da origen a como máximo una reseña. El campo id_reserva tiene índice único en MongoDB para garantizar esto.
reseñas → votos_utilidad	1 : N	Una reseña puede recibir votos de muchos usuarios. La relación se materializa por referencia desde votos_utilidad hacia el _id de la reseña.
CLIENTE → votos_utilidad	1 : N	Un cliente puede votar muchas reseñas, pero solo una vez por reseña. El índice único { id_resena, id_usuario } garantiza esto.
reseñas → respuesta_hotel	1 : 0..1	Cada reseña puede tener como máximo una respuesta oficial del hotel. Se almacena como subdocumento embebido dentro del documento de la reseña.

2.c Análisis de esquema de asociación (Anexo C)

Relación	Simplicity	Go Together	Query Atomicity	Cardinality	Doc Growth	Workload	Decisión
reseñas ↔ respuesta_hotel	Embed	Embed	Embed	Embed (0..1)	Embed (acotado)	Embed	EMBEBIDO
reseñas ↔ HOTEL (id_hotel)	Ref	Ref	Ref	Ref (1:N)	—	—	REFERENCIADO
reseñas ↔ CLIENTE (id_cliente)	Ref	Ref	Ref	Ref (1:N)	—	—	REFERENCIADO
reseñas ↔ RESERVA (id_reserva)	Ref	—	—	Ref (1:1 único)	—	—	REFERENCIADO
reseñas ↔ votos_utilidad	Ref	Parcial	Ref	Ref (crece sin límite)	Ref (crece sin límite)	Ref (alta escritura)	REFERENCIADO (colección aparte)

Nota sobre la respuesta del hotel: el subdocumento respuesta_hotel se embebe porque se lee siempre junto con la reseña (RF4, RF6, RFC3), tiene cardinalidad máxima de uno, tamaño acotado (máx. ~1.000 caracteres) y no crece indefinidamente. Embeber aquí es la decisión correcta tanto en términos de rendimiento como de simplicidad del código APEX.

Nota sobre votos_utilidad: se separan en colección propia porque crecen sin límite (~500 registros nuevos cada día), requieren control de unicidad con índice compuesto, y se escriben con una frecuencia y en momentos completamente independientes de la reseña. Mantenerlos embebidos como array causaría problemas de tamaño de documento y conflictos de escritura concurrente.

2.d Estructura de documentos en JSON (Anexo D)

Colección reseñas — esquema con subdocumento embebido:

```
{
  "_id": ObjectId("64alb2c3d4e5f6789012abcd"),

  // Referencias a Oracle
  "id_hotel": 1,      // HOTEL.ID_HOTEL
  "id_cliente": 42,   // CLIENTE.ID_CLIENTE
  "id_reserva": 107,  // RESERVA.ID_RESERVA (índice único)

  // Contenido de la reseña
  "calificacion": 5,
  "texto": "El servicio superó las expectativas. Habitaciones muy limpias.",
  "estado": "publicada", // "publicada" | "eliminada"

  // Metadatos
  "fecha_creacion": ISODate("2024-03-15T10:30:00Z"),
  "fecha_modificacion": ISODate("2024-03-16T08:15:00Z"),
  "destacada": true,
  "votos_utilidad": 8, // contador desnormalizado

  // Subdocumento EMBEBIDO: respuesta oficial del hotel
  "respuesta_hotel": {
    "texto": "Gracias por su estadia, lo esperamos pronto.",
    "fecha": ISODate("2024-03-17T09:00:00Z"),
    "id_admin": "admin_hotel1"
  }
}
```

Colección votos_utilidad — esquema referenciado:

```
// Cada documento registra UN voto de UN usuario sobre UNA reseña.
// El índice único { id_resena: 1, id_usuario: 1 } garantiza
// que no haya duplicados (un usuario, un voto por reseña).

{
  "_id": ObjectId("64b9f3a1e2d5c6789abcdef0"),
  "id_resena": ObjectId("64alb2c3d4e5f6789012abcd"), // ref → reseñas._id
  "id_usuario": 5, // ref → CLIENTE.ID_CLIENTE
  "fecha_voto": ISODate("2024-03-16T14:22:00Z")
}
```

1.c Creación de Colecciones en MongoDB

A continuación se presenta el script de creación de las colecciones e índices en MongoDB:

```
use("dann_alpes_resenas");

// 1. COLECCIÓN: resenas
db.createCollection("resenas");

// 2. COLECCIÓN: votos_utilidad
db.createCollection("votos_utilidad");

// Índices de resenas

// Consultas por hotel (RF4, RFC1, RFC2, RFC3)
db.resenas.createIndex({ id_hotel: 1, fecha_creacion: -1 });

// Historial del cliente (RF6)
db.resenas.createIndex({ id_cliente: 1, fecha_creacion: -1 });
```

```
// Unicidad: un cliente solo puede reseñar una reserva una vez (RF1)
db.resenas.createIndex(
  { id_reserva: 1 },
  { unique: true, sparse: true }
);

// Reseña destacada por hotel (RF9)
db.resenas.createIndex(
  { id_hotel: 1, destacada: 1 },
  { sparse: true }
);

// Índices de votos_utilidad

// Un usuario solo puede votar una reseña una vez (RF5)
db.votos_utilidad.createIndex(
  { id_resena: 1, id_usuario: 1 },
  { unique: true }
);
```

1.d Esquemas de Validación

A continuación se presentan los esquemas de validación JSON Schema aplicados a cada colección:

```
use("dann_alpes_resenas");
```

```
// VALIDACIÓN: resenas
db.runCommand({
  collMod: "resenas",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [
        "id_hotel", "id_cliente", "id_reserva",
        "calificacion", "texto", "estado", "fecha_creacion"
      ],
      properties: {
        id_hotel: { bsonType: "int", minimum: 1 },
        id_cliente: { bsonType: "int", minimum: 1 },
        id_reserva: { bsonType: "int", minimum: 1 },
        calificacion: { bsonType: "int", minimum: 1, maximum: 5 },
        texto: { bsonType: "string", minLength: 10, maxLength: 2000 },
        estado: { bsonType: "string", enum: ["publicada", "eliminada"] },
        fecha_creacion: { bsonType: "date" },
        fecha_modificacion: { bsonType: "date" },
        fecha_eliminacion: { bsonType: "date" },
        eliminado_por: { bsonType: "string", enum: ["cliente", "administrador"] },
        destacada: { bsonType: "bool" },
        votos_utilidad: { bsonType: "int", minimum: 0 },
        respuesta_hotel: {
          bsonType: "object",
          required: ["texto", "fecha"],
          properties: {
            texto: { bsonType: "string", minLength: 5, maxLength: 1000 },
            fecha: { bsonType: "date" },
            id_admin: { bsonType: "string" }
          }
        }
      }
    },
    additionalProperties: true
  }
},
  validationLevel: "strict",
  validationAction: "error"
});

// VALIDACIÓN: votos_utilidad
db.runCommand({
  collMod: "votos_utilidad",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_resena", "id_usuario", "fecha_voto"],
```

```
    properties: {
      id_resena: { bsonType: "objectId" },
      id_usuario: { bsonType: "int", minimum: 1 },
      fecha_voto: { bsonType: "date" }
    },
    additionalProperties: false
  },
  validationLevel: "strict",
  validationAction: "error"
});
```

4. Sentencias MongoDB – Requerimientos de Consulta (RFC)

RFC1 – Top 10 hoteles por calificación promedio en un período

Esta consulta opera exclusivamente sobre la colección reseñas. Los atributos relevantes son id_hotel, calificacion, estado y fecha_creacion. El pipeline tiene cuatro etapas:

\$match: filtra para quedarse solo con reseñas publicadas dentro del rango de fechas solicitado.

\$group: agrupa por id_hotel y calcula el promedio de calificacion con \$avg, el conteo de reseñas con \$sum:1 y la suma de votos de utilidad.

\$sort: ordena descendente por calificación promedio. En caso de empate, desempata por número de reseñas.

\$limit: retorna los 10 primeros resultados.

```
db.resenas.aggregate([
  {
    $match: {
      estado: "publicada",
      fecha_creacion: {
        $gte: ISODate("2024-01-01"),
        $lte: ISODate("2024-12-31")
      }
    }
  },
  {
    $group: {
      _id: "$id_hotel",
      calificacion_promedio: { $avg: "$calificacion" },
      total_resenas: { $sum: 1 },
      total_votos: { $sum: "$votos_utilidad" }
    }
  },
  { $sort: { calificacion_promedio: -1, total_resenas: -1 } },
  { $limit: 10 },
  {
    $project: {
      _id: 0,
      id_hotel: "$_id",
      calificacion_promedio: { $round: ["$calificacion_promedio", 2] },
      total_resenas: 1,
      total_votos: 1
    }
  }
])
```

RFC2 – Evolución de la reputación de un hotel mes a mes

Opera también sobre reseñas. Los atributos usados son id_hotel, calificacion, estado y fecha_creacion. La lógica es sencilla: se filtra por hotel y año, se agrupa por mes usando el operador \$month, y se calcula el promedio de ese mes. El resultado permite trazar la curva de reputación a lo largo del año.

Por qué \$month: es el operador nativo de MongoDB para extraer el componente de mes de un campo de fecha, lo que evita convertir fechas a strings o hacer cálculos auxiliares.

```
// Cambiar ID_HOTEL y AÑO según la consulta requerida
db.resenas.aggregate([
  {
    $match: {
      estado: "publicada",
      id_hotel: 1,
      fecha_creacion: {
        $gte: ISODate("2024-01-01"),
        $lte: ISODate("2024-12-31")
      }
    }
  }
])
```



```

    }
  },
  {
    $group: {
      _id: { mes: { $month: "$fecha_creacion" } },
      calificacion_promedio: { $avg: "$calificacion" },
      total_resenas: { $sum: 1 }
    }
  },
  { $sort: { "_id.mes": 1 } },
  {
    $project: {
      _id: 0,
      mes: "$_id.mes",
      calificacion_promedio: { $round: ["$calificacion_promedio", 2] },
      total_resenas: 1
    }
  }
}
})

```

RFC3 – Perfil comparativo de hoteles por ciudad

Esta es la consulta más compleja. Devuelve, para cada hotel de una ciudad: calificación promedio, total de reseñas, porcentaje de reseñas con respuesta del administrador, porcentaje de reseñas destacadas, y un flag que indica si el hotel está por debajo del promedio de la ciudad.

Por qué \$facet: permite ejecutar dos ramas de agregación en un solo pase sobre la colección. Una rama calcula las métricas por hotel; la otra calcula el promedio global de la ciudad. Después se combinan para agregar el flag bajo_promedio. Sin \$facet habría que hacer dos consultas separadas y combinarlas en el lado del cliente.

Los IDs de los hoteles de la ciudad se proveen desde Oracle vía APEX (son los hoteles asociados a la ciudad seleccionada en la interfaz).

```

// HOTELES_CIUADAD: array de id_hotel que pertenecen a la ciudad seleccionada
// (obtenido desde Oracle en APEX y pasado como parámetro)
db.resenas.aggregate([
  {
    $match: {
      estado: "publicada",
      id_hotel: { $in: [1, 2] } // hoteles de la ciudad
    }
  },
  {
    $facet: {
      por_hotel: [
        {
          $group: {
            _id: "$id_hotel",
            calificacion_promedio: { $avg: "$calificacion" },
            total_resenas: { $sum: 1 },
            con_respuesta: {
              $sum: { $cond: [{ $ifNull: ["$respuesta_hotel", false] }, 1, 0] }
            },
            destacadas: {
              $sum: { $cond: [{ $eq: ["$destacada", true] }, 1, 0] }
            }
          }
        }
      ],
      {
        $project: {
          _id: 0,
          id_hotel: "$_id",
          calificacion_promedio: { $round: ["$calificacion_promedio", 2] },
          total_resenas: 1,
          pct_respuesta: {
            $round: [{ $multiply:
              [{ $divide: ["$con_respuesta", "$total_resenas"] }, 100] }, 1]
          }
        }
      }
    }
  }
])

```

```

    },
    pct_destacadas: {
      $round: [{ $multiply:
        [{ $divide: ["$destacadas", "$total_resenas"] }, 100] }, 1]
    }
  }
],
promedio_ciudad: [
  { $group: { _id: null, prom: { $avg: "$calificacion" } } }
]
}
},
{ $unwind: "$por_hotel" },
{
  $addFields: {
    "por_hotel.promedio_ciudad": {
      $round: [{ $arrayElemAt: ["$promedio_ciudad.prom", 0] }, 2]
    },
    "por_hotel.bajo_promedio": {
      $lt: [
        "$por_hotel.calificacion_promedio",
        { $arrayElemAt: ["$promedio_ciudad.prom", 0] }
      ]
    }
  }
},
{ $replaceRoot: { newRoot: "$por_hotel" } },
{ $sort: { calificacion_promedio: -1 } }
])

```

6. Escenarios de Prueba

6.1 Pruebas de esquema de validación (RF1 y RF2)

El script 05_escenarios_prueba.js ejecuta siete pruebas contra el esquema JSON Schema de validación. Cada una intenta insertar un documento que viola una regla distinta. Las primeras cinco deben fallar; la sexta debe pasar; la séptima verifica el índice único.

#	Descripción	Regla que se prueba	Resultado esperado
1	Reseña sin campo calificacion	Campo requerido ausente	Error de validación (document failed validation)
2	Calificación con valor 7	Valor fuera del rango máximo (1–5)	Error de validación
3	Estado = 'pendiente'	Valor no contemplado en el enum	Error de validación
4	Texto de 3 caracteres	minLength = 10	Error de validación
5	Reseña sin fecha_creacion	Campo requerido ausente	Error de validación
6	Documento completamente válido	Ninguna violación	Inserción exitosa, _id asignado
7	Voto duplicado del mismo usuario sobre la misma reseña	Índice único { id_resena, id_usuario }	Error de clave duplicada (E11000)

6.2 Datos de prueba para consultas RFC (Script 03)

El script 03_poblar_datos.js carga 29 reseñas distribuidas en seis hoteles de tres ciudades (Bogotá, Medellín, Cartagena), con fechas repartidas en varios meses de 2024. Esta distribución garantiza que las tres consultas RFC retornen resultados no triviales: diferencias de calificación entre hoteles y ciudades, evolución mensual visible para RFC2, y hoteles que caen por debajo del promedio de su ciudad en RFC3.

Hotel ID	Ciudad	Reseñas	Cal. prom.	Con respuesta	Destacadas	2 reseñas eliminadas
1	Bogotá	6	4,67	4 (67 %)	1	—
2	Bogotá	4	3,25	2 (50 %)	0	—
3	Medellín	5	4,80	3 (60 %)	1	—
4	Medellín	4	3,50	1 (25 %)	0	—
5	Cartagena	5	4,60	3 (60 %)	1	—
6	Cartagena	3	3,67	1 (33 %)	0	—
1 y 2	Bogotá	—	—	—	—	2 reseñas en estado=eliminada (RF3, RF6, RF8)

Archivo	Descripción
01_crear_colecciones.js	Crea las colecciones reseñas y votos_utilidad con todos los índices necesarios.
02_validacion_esquemas.js	Aplica JSON Schema validation a ambas colecciones con nivel strict.
03_poblar_datos.js	Inserta 29 reseñas de prueba distribuidas en 6 hoteles y 3 ciudades, más 10 votos de utilidad.
04_consultas_RFC.js	Contiene los tres pipelines de agregación (RFC1, RFC2, RFC3) listos para ejecutar.
05_escenarios_prueba.js	Ejecuta las 7 pruebas de validación de esquema e índice único.

Capturas de Pantalla

Crear reseña exitosa

Crear Reseña

Nueva Reseña

Hotel:

ID del hotel

ID de tu reserva:

ID de reserva completada

Calificación:

☆ ☆ ☆ ☆ ☆

Tu experiencia:

Cuéntanos tu experiencia (mínimo 10 caracteres)

Publicar reseña

✓ Reseña publicada correctamente.

Editar reseña

Editar Reseña

Editar reseña

Calificación (1-5):

5

Texto:

Excelente hotel

Guardar

Cancelar

Eliminar reseña

Hotel ID: 1

eliminada

★★★★★ 2026-05-22

Excelente servicio. Me gustó mucho el brunch

0 votos útiles | Sin respuesta del hotel

Ver reseñas del hotel

Reseñas

Destacada

★★★★☆ 2026-05-22

No me gustó nada

0 personas encontraron útil esta resena

Util

Respuesta del hotel:

lástima

★★★★★ 2026-05-22

Excelente hotel

0 personas encontraron útil esta resena

Util

★★★★★ 2024-09-03

Hotel de lujo con atención personalizada. Volvería sin dudarlo.

11 personas encontraron útil esta resena

Util

Respuesta del hotel:

Un placer recibirlo, lo esperamos de vuelta.

Votar útil

★★★★★

2024-05-08

Una experiencia magnífica de principio a fin. El spa es de primera.

16 personas encontraron útil esta reseña

Util

Respuesta del hotel:

Gracias por elegirnos para su descanso.

★★★★☆

2024-03-20

Muy buen hotel en general, solo el ruido del exterior molestó un poco.

6 personas encontraron útil esta reseña

Util

★★★★★

2024-02-05

Ubicación perfecta, desayuno delicioso y las vistas son increíbles.

9 personas encontraron útil esta reseña

Util

Respuesta del hotel:

Nos alegra que haya disfrutado su estadía.

★★★★★

2024-01-10

Excelente servicio al cliente y habitaciones muy cómodas.

12 personas encontraron útil esta reseña

Util

Historial Propio

Mis Reseñas

Mis reseñas

Ordenar por:

17 Fecha

Hotel

Hotel ID: 1

★★★★★ 2026-05-22

Excelente hotel

0 votos utiles | Sin respuesta del hotel

Editar

Eliminar

publicada

Hotel ID: 1

★☆☆☆☆ 2026-05-22

No me gustó nada

0 votos utiles | Tiene respuesta del hotel

Editar

Eliminar

publicada

Responder Reseña

Responder resena

Escribe la respuesta oficial del hotel...

Guardar respuesta

Cancelar

App 14866 Page 16 Session Debug Quick Edit Customize

Eliminar reseña desde admin

iacademy2.oracle.com dice
Eliminar esta resena por violacion de politicas?
Cancelar Aceptar

Responder Destacar Eliminar

★★★★☆
Muy buen hotel en general, solo el ruido del exterior molestó un poco.
Responder Destacar Eliminar

2024-03-20

★★★★★
Ubicación perfecta, desayuno delicioso y las vistas son increíbles.
Respuesta actual: Nos alegra que haya disfrutado su estadía.
Responder Destacar Eliminar

2024-02-05

★★★★★
Excelente servicio, habitaciones muy limpias y cómodas. El personal muy atento.
Respuesta actual: Gracias por su visita, lo esperamos pronto.
Responder Destacar Eliminar

2024-01-10

Responder resena

Escribe la respuesta oficial del hotel...

Guardar respuesta Cancelar

App 14866 Page 16 Session Debug Quick Edit Customize

Destacar reseña

Panel Admin Reseñas

New

Hotel ID:

Buscar

★★★★★ Destacada

Excelente hotel

Responder

Quitar destaque

Eliminar

Top Hoteles

Top Hoteles

Desde:

Hasta:

Consultar

#	Hotel ID	Calificacion Promedio	Total Resenas	Total Votos
1	Hotel 3	★★★★★ 4.8	5	73
2	Hotel 1	★★★★★ 4.67	6	60
3	Hotel 5	★★★★ 4.4	5	80
4	Hotel 6	★★★★ 3.67	3	22
5	Hotel 4	★★★★ 3.5	4	23
6	Hotel 2	★★★ 3.25	4	34

Evolución Reputación

New

Hotel ID:

Año:

Consultar

Hotel 1 - Año 2024

Mes	Promedio	Resenas	Tendencia
Enero	5 / 5	1	
Febrero	5 / 5	1	
Marzo	4 / 5	1	
Mayo	5 / 5	1	
Julio	4 / 5	1	
Septiembre	5 / 5	1	

Ciudad

Bogota

Comparativo Ciudad

Comparativo por ciudad

Consultar

Promedio de la ciudad: 4.25 / 5

Hotel	Cal. Promedio	Total Resenas	% Con Respuesta	% Destacadas	Estado
Hotel 1	4.25	8	62.5%	12.5%	OK

Pruebas de validación

```

// PRUEBA 1: Sin calificación
try {
  db.resenas.insertOne({
    id_hotel: 1, id_cliente: 1, id_reserva: 999,
    texto: "Esta resena no tiene calificacion.",
    estado: "publicada", fecha_creacion: new Date()
  });
  print("ERROR: No debio insertarse");
} catch(e) { print("OK - Rechazado: " + e.message.substring(0,80)); }

// PRUEBA 2: Calificación fuera de rango
try {
  db.resenas.insertOne({
    id_hotel: 1, id_cliente: 1, id_reserva: 998,
    calificacion: 7, texto: "Calificación invalida.",
    estado: "publicada", fecha_creacion: new Date()
  });
  print("ERROR: No debio insertarse");
} catch(e) { print("OK - Rechazado: " + e.message.substring(0,80)); }

// PRUEBA 3: Estado invalido
try {
  db.resenas.insertOne({
    id_hotel: 1, id_cliente: 1, id_reserva: 997,
    calificacion: 4, texto: "Estado no permitido en este documento.",
    estado: "pendiente", fecha_creacion: new Date()
  });
  print("ERROR: No debio insertarse");
} catch(e) { print("OK - Rechazado: " + e.message.substring(0,80)); }

print("=== Pruebas completadas ===");
< OK - Rechazado: Document failed validation
< OK - Rechazado: Document failed validation
< OK - Rechazado: Document failed validation
< === Pruebas completadas ===

```